Member Contributions

- **Supriyo Ghosh:** Implemented the **core loader logic**, including ELF parsing, SIGSEGV-based on-demand paging, and page fault handling using `mmap()`. Also tested and verified memory mapping and fragmentation outputs.

- **Hariom Tiwari:** Handled **program execution flow**, cleanup, and final report generation. Managed ELF validation, Makefile setup, and documentation, ensuring smooth compilation, testing, and submission.

Github Link: https://github.com/itz-hariom/OS_Assignment_4

SimpleSmartLoader: Implementation

1. Overview

SimpleSmartLoader is a user-level ELF loader that implements **on-demand paging**.
It does not map all ELF segments upfront — instead, it catches **SIGSEGV** when the program accesses an unmapped address, maps that single page using `mmap()`, loads the required bytes from the ELF, and resumes execution.

The loader also reports:

- Total Page Faults
- Total Page Allocations
- Total Internal Fragmentation (KB)

2. System Architecture

Main Components:

1. **ELF Loader** – Reads and validates ELF32 headers using `Elf32_Ehdr`

and `Elf32_Phdr`.

2. **Segment Table** – Stores details of each `PT_LOAD` segment (offset, vaddr, filesz, memsz, flags).
3. **SIGSEGV Handler** – Handles page faults, maps one page, copies file data or zeroes BSS, updates stats.
4. **Runner** – Invokes the ELF entry point and executes the target program.
5. **Reporter** – Prints total page faults, allocations, and internal fragmentation after program exit.

3. Working Mechanism

1. The ELF file is loaded into memory and verified (magic bytes, 32-bit, little-endian, ET_EXEC, EM_386).
2. The **signal handler** is registered via `sigaction(SIGSEGV, ...)`.
3. On every page fault:
    - Find which segment contains the faulting address.
    - Map a 4KB page at that address using `mmap(MAP_FIXED)`.
    - Copy data from ELF (or zero-fill if `.bss`).
    - Apply final protections via `mprotect()` based on segment flags.
    - Update counters for faults, allocations, and fragmentation.
4. The loader calls the ELF's `_start` function to begin execution.
5. When the program finishes, the loader prints the final report.

4. Example Output

fib.c:

```
User _start return value = 102334155

Total Page Faults: 1
```

```
Total Page Allocations: 1

Total Internal Fragmentation: 3 KB
```

sum.c:

```
User _start return value = 2048

Total Page Faults: 3

Total Page Allocations: 3

Total Internal Fragmentation: 7 KB
```

5. Limitations & Future Work

- Only supports **32-bit ELF executables (ELFCLASS32, EM_386)**.
- Uses `mmap()` and `memcpy()` inside the signal handler (not strictly async-signal-safe).
- Does not handle relocations or dynamic linking.
- Future scope: support 64-bit binaries, safer fault handling, and enhanced statistics collection.

6. Key System Calls

`open`, `read`, `lseek`, `mmap`, `mprotect`, `sigaction`, `memcpy`, `memset`, `close`, `exit`.

***Result**:* The loader successfully demonstrates **demand paging** using SIGSEGV, mapping pages dynamically, and producing accurate memory statistics for each test program.